

Academic Project Review 2

Hotel Reservation and Guest Services Management System

Normalization, constraints and SQL — the database, built and tested in MySQL 8

Name — Dandigam Ritheek

Course — AI-ML

Institution — Woxsen University

Academic Year — 2026

Normalization

Starting point: one flat booking table

The flat table (everything in one place)

- guest name, email, phone — repeated on every booking
- room number, room type, capacity, nightly rate — repeated per booking
- check-in / check-out dates, adults, children
- service lines, invoice total, payment — repeating groups per stay

What goes wrong

- **Update anomaly** — change a room type's capacity and every booking row must be edited
- **Insert anomaly** — a new room type or guest cannot exist until a booking does
- **Delete anomaly** — deleting a guest's only booking deletes the guest
- **Repeating groups** — a stay has many services and many payments (breaks 1NF)

Normalization

From 1NF to 3NF

- **1NF — atomic values**

Every column holds one value. Service lines and payments leave the booking row and get their own tables (service_request, service_charge, payment). Statuses are single-valued ENUMs.

- **2NF — no partial dependencies**

Every table has a single-column surrogate key (reservation_id, room_id...), so no attribute can depend on only part of a key. Guest, room, rate and staff details leave the booking table.

- **3NF — no transitive dependencies**

room stores type_id, not capacity. reservation stores rate_plan_id, not the rate. Money collected lives in payment, not in invoice. 12 tables result.

Functional dependencies per table (1 of 2)

Table	Functional dependencies	Result
room_type	type_id → type_name, capacity, description type_name → type_id (candidate key)	BCNF
room	room_id → room_number, type_id, status room_number → room_id (candidate key)	BCNF
rate_plan	rate_plan_id → type_id, plan_name, nightly_rate, description	BCNF
guest	guest_id → full_name, email, phone email → guest_id (candidate key)	BCNF
staff	staff_id → full_name, role	BCNF
reservation	reservation_id → guest_id, room_id, rate_plan_id, reservation_status, planned_check_in, planned_check_out, adults, children, created_at	BCNF

Functional dependencies per table (2 of 2)

Table	Functional dependencies	Result
stay	stay_id → reservation_id, actual_check_in, actual_check_out reservation_id → stay_id (1:0..1, candidate key)	BCNF
housekeeping_task	task_id → room_id, stay_id, task_type, assigned_to, task_status, scheduled_at, completed_at stay_id → room_id (only when stay_id is present) †	3NF exception †
service_request	service_request_id → stay_id, service_name, requested_at, status	BCNF
service_charge	charge_id → stay_id, service_request_id, description, amount, charged_at service_request_id → stay_id (only when present) †	3NF exception †
invoice	invoice_id → stay_id, invoice_date, subtotal, tax, total stay_id → invoice_id (1:1, candidate key)	BCNF
payment	payment_id → invoice_id, payment_date, amount, payment_method, payment_status	BCNF

Normalization

Design decisions and honest trade-offs

- **Stay is separate from Reservation**

A reservation exists before arrival and can be cancelled or a no-show. A stay holds the actual check-in/out and exists only if the guest arrives (1 : 0..1).

- **Invoice totals are stored**

Derivable from the stay and its charges, but kept as a billing snapshot so a closed invoice never changes when rates change.

- **Staff is its own table**

The ER diagram showed assigned_to as a plain field. Making it a foreign key stops staff names and roles repeating on every task.

- **Two controlled exceptions to 3NF †**

service_charge.stay_id and housekeeping_task.room_id stay stored because the optional link may be absent. Triggers reject any row where they disagree.

Data dictionary (1 of 3)

room_type <i>Category of room: capacity and description.</i>		
type_id	INT	PK
type_name	VARCHAR(50)	UNIQUE
capacity	INT	
description	VARCHAR(255)	NULL

rate_plan <i>Priced plan for one room type.</i>		
rate_plan_id	INT	PK
type_id	INT	FK → room_type
plan_name	VARCHAR(50)	
nightly_rate	DECIMAL(10,2)	
description	VARCHAR(255)	NULL

room <i>Physical room, its type and live status.</i>		
room_id	INT	PK
room_number	VARCHAR(10)	UNIQUE
type_id	INT	FK → room_type
status	ENUM(4)	DEFAULT available

guest <i>Guest contact details.</i>		
guest_id	INT	PK
full_name	VARCHAR(100)	
email	VARCHAR(100)	UNIQUE
phone	VARCHAR(20)	NULL

Data dictionary (2 of 3)

staff <i>Hotel staff who can be assigned housekeeping tasks.</i>		
staff_id	INT	PK
full_name	VARCHAR(100)	
role	VARCHAR(50)	

reservation <i>A planned booking: who, which room, which plan, which dates.</i>		
reservation_id	INT	PK
guest_id	INT	FK → guest
room_id	INT	FK → room
rate_plan_id	INT	FK → rate_plan
reservation_status	ENUM(5)	DEFAULT booked
planned_check_in	DATE	
planned_check_out	DATE	
adults	INT	DEFAULT 1
children	INT	DEFAULT 0
created_at	TIMESTAMP	NULL · DEFAULT CURRENT_TIMESTAMP

stay <i>The actual check-in/check-out against a reservation.</i>		
stay_id	INT	PK
reservation_id	INT	FK → reservation · UNIQUE
actual_check_in	DATETIME	
actual_check_out	DATETIME	NULL

housekeeping_task <i>Cleaning or maintenance job for a room.</i>		
task_id	INT	PK
room_id	INT	FK → room
stay_id	INT	FK → stay · NULL
task_type	VARCHAR(50)	
assigned_to	INT	FK → staff · NULL
task_status	ENUM(3)	DEFAULT pending
scheduled_at	DATETIME	
completed_at	DATETIME	NULL

Data dictionary (3 of 3)

service_request <i>A guest's in-stay request for a service.</i>		
service_request_id	INT	PK
stay_id	INT	FK → stay
service_name	VARCHAR(100)	
requested_at	DATETIME	NULL · DEFAULT CURRENT_TIMESTAMP
status	ENUM(3)	DEFAULT requested

invoice <i>Bill for a stay: subtotal, tax, total.</i>		
invoice_id	INT	PK
stay_id	INT	FK → stay · UNIQUE
invoice_date	DATE	NULL · DEFAULT CURRENT_DATE
subtotal	DECIMAL(10,2)	DEFAULT 0.00
tax	DECIMAL(10,2)	DEFAULT 0.00
total	DECIMAL(10,2)	DEFAULT 0.00
status	ENUM(3)	DEFAULT open

service_charge <i>Billable amount for a service (or ad-hoc charge).</i>		
charge_id	INT	PK
stay_id	INT	FK → stay
service_request_id	INT	FK → service_request · NULL
description	VARCHAR(255)	NULL
amount	DECIMAL(10,2)	
charged_at	DATETIME	NULL · DEFAULT CURRENT_TIMESTAMP

payment <i>One payment against an invoice.</i>		
payment_id	INT	PK
invoice_id	INT	FK → invoice
payment_date	DATETIME	NULL · DEFAULT CURRENT_TIMESTAMP
amount	DECIMAL(10,2)	
payment_method	ENUM(4)	
payment_status	ENUM(3)	DEFAULT success

Implementation

Constraints, indexes and views in the schema

12

tables

12

primary keys

14

foreign keys

5

UNIQUE

8

CHECK

7

ENUM columns

16

DEFAULT values

3

secondary indexes

10

triggers

8

views

- **Keys:** every table has an auto-increment primary key; foreign keys link every relationship in the ER diagram.
- **Indexes:** on reservation(room_id, dates) for availability search, stay(check-in, check-out) and guest(full_name).
- **NOT NULL / CHECK / DEFAULT:** positive capacity, non-negative amounts, valid date order, sensible defaults (status = booked / available / pending).
- **Triggers** cover rules a CHECK cannot express because they look across rows or tables.

Eight business rules, each tested by breaking it

Rule	Enforced by	Result when violated
No overlapping room bookings	Trigger	Room is already booked for an overlapping date range.
Occupants within room capacity	Trigger	Occupant count exceeds room type capacity.
Check-out after check-in	CHECK	Check constraint 'chk_dates' is violated.
Charge amounts not negative	CHECK	Check constraint 'chk_charge_nonneg' is violated.
Payments cannot exceed invoice total	Trigger	Payment would exceed the invoice total.
Rate plan belongs to the room's type	Trigger	Rate plan does not belong to this room type.
Charge's stay matches its request's stay	Trigger	Service charge stay does not match its service request.
Task's room matches its stay's room	Trigger	Housekeeping task room does not match the stay's room.

Query catalogue: joins, subqueries, aggregation, views

Query	Technique	Answers
Q1	INNER JOIN, 5 tables	Every reservation with guest, room, type and rate
Q2	LEFT JOIN	All rooms with current guest (NULL if vacant)
Q3	JOIN + date function	Invoice breakdown per stay
Q4	Nested subquery (NOT IN)	Rooms free for a requested date range
Q5	Subquery in HAVING	Stays spending above the average on services
Q6	GROUP BY + aggregates	Reservations and average rate per room type
Q7	GROUP BY + HAVING	Housekeeping workload per staff member
Q8	Scalar subqueries	Revenue collected and service revenue
Q9	Subquery ratio	Cancellation rate
Q10–Q12	Views	Occupancy, guest history, housekeeping status
Q13	EXPLAIN	Availability lookup uses the composite index

All 8 reporting views and every query above are in **05_queries.sql** and **04_views_reports.sql**; actual output is in **queries_output.txt**.

Joins and subqueries — real output

Q2 — LEFT JOIN

```
SELECT r.room_number, rt.type_name, r.status, g.full_name AS current_guest
FROM room r
JOIN room_type rt ON r.type_id = rt.type_id
LEFT JOIN reservation res ON res.room_id = r.room_id AND res.reservation_status = 'checked_in'
LEFT JOIN guest g ON res.guest_id = g.guest_id
ORDER BY r.room_number;
```

room_number	type_name	status	current_guest
101	Standard	available	NULL
102	Standard	available	NULL
103	Standard	occupied	Arjun Mehta
201	Deluxe	available	NULL
202	Deluxe	available	NULL
301	Suite	available	NULL
302	Suite	maintenance	NULL

Q4 — Nested subquery: rooms free 11–12 Oct 2026

```
SELECT r.room_number, rt.type_name
FROM room r
JOIN room_type rt ON r.type_id = rt.type_id
WHERE r.status <> 'maintenance'
AND r.room_id NOT IN (
  SELECT room_id FROM reservation
  WHERE reservation_status NOT IN ('cancelled','no_show')
  AND '2026-10-11' < planned_check_out
  AND '2026-10-12' > planned_check_in)
ORDER BY r.room_number;
```

room_number	type_name
101	Standard
102	Standard
103	Standard
202	Deluxe
301	Suite

Aggregation, ratios and index use — real output

Q6 — GROUP BY

```
SELECT rt.type_name, COUNT(res.reservation_id) AS reservations,
       ROUND(AVG(rp.nightly_rate), 2) AS avg_nightly_rate
FROM room_type rt
LEFT JOIN room r ON r.type_id = rt.type_id
LEFT JOIN reservation res ON res.room_id = r.room_id
LEFT JOIN rate_plan rp ON res.rate_plan_id = rp.rate_plan_id
GROUP BY rt.type_name;
```

type_name	reservations	avg_nightly_rate
Deluxe	1	4200.00
Standard	3	2366.67
Suite	1	7500.00

Q9 — Subquery ratio

```
SELECT ROUND(100 * (SELECT COUNT(*) FROM reservation
                    WHERE reservation_status IN ('cancelled','no_show'))
            / (SELECT COUNT(*) FROM reservation), 2) AS cancellation_pct;
```

cancellation_pct
20.00

Q8 — Scalar subqueries

```
SELECT (SELECT COALESCE(SUM(amount),0) FROM payment WHERE payment_status='success') AS revenue_collected,
       (SELECT COALESCE(SUM(amount),0) FROM service_charge) AS service_revenue;
```

revenue_collected	service_revenue
4410.00	2800.00

Q13 — EXPLAIN: the index is used

table	type	possible_keys	key	rows	Extra
reservation	range	idx_reservation_dates	idx_reservation_dates	1	Using where; Using index

Prototype

Live database: built, loaded and tested

Rows loaded per table (sample data)

Table	Rows
guest	5
housekeeping_task	3
invoice	1
payment	1
rate_plan	5
reservation	5
room	7
room_type	3
service_charge	3
service_request	3
staff	3
stay	2

Built and verified in MySQL 8

- 12 tables, 8 views and 10 triggers created from 4 scripts, run in order
- All 13 queries execute and return correct results
- 8 of 8 deliberate rule violations rejected, no data changed
- Python CLI (Review 3) runs end-to-end on this same database

Example: overlapping booking attempted

```
INSERT INTO reservation (guest_id,room_id,rate_plan_id,planned_check_in,planned_check_out,adults,children)
VALUES (2,3,1,'2026-09-22','2026-09-26',1,0)
```

ERROR 1644 (45000): Room is already booked for an overlapping date range.

Thank You

Questions & Technical Discussion